VolFx

Quick Guide

• dev by NullTale ₊⁺

VolFx is customizable selective post-processing and buffer based rendering system.
that also allows to build a custom scene architexture for visual effects creation

Documentation Content

The documentation is divided into three main sections:

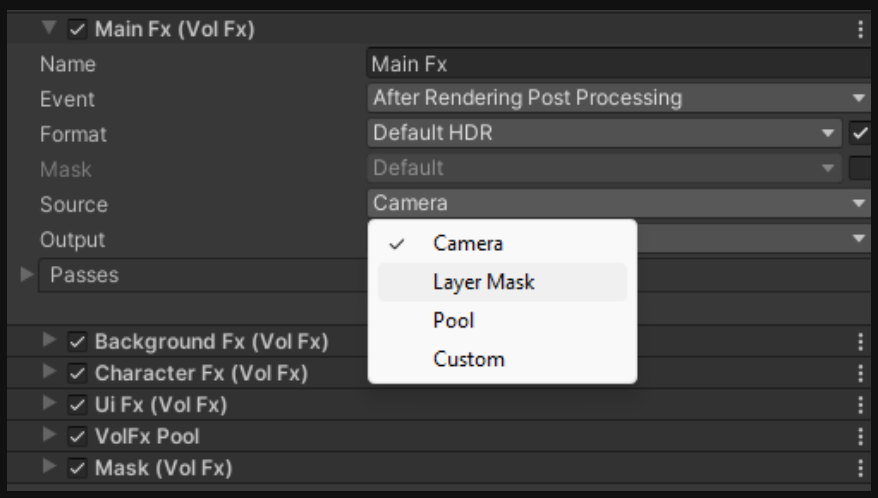
1. **Creating and configuring a rendering processing module**
2. **Setting up selective effect application using LayerMask**
3. **Implementing a simple custom visual effect**

* More detailed information and usage examples can be found in the **Project Samples**.

* To use this Asset you need basic understanding of what Urp is and how to configure it for your project, it can be found in the official documentation. Debugging and building PostProcessing Chain is hard to do blindly, it is highly recommended to use Frame Debugger
(more information about rendering process can be found in the official Unity documentation)

General

VolFx consists of **modules (Render Passes)** that can process different sources, such as:



- Camera content
- Global Texture
- Objects collected by **LayerMask**

* this allows you to: control the scene and display groups of objects, process textures for effects (light maps, pattern animations, height maps, etc.)

Creating a Processing Module

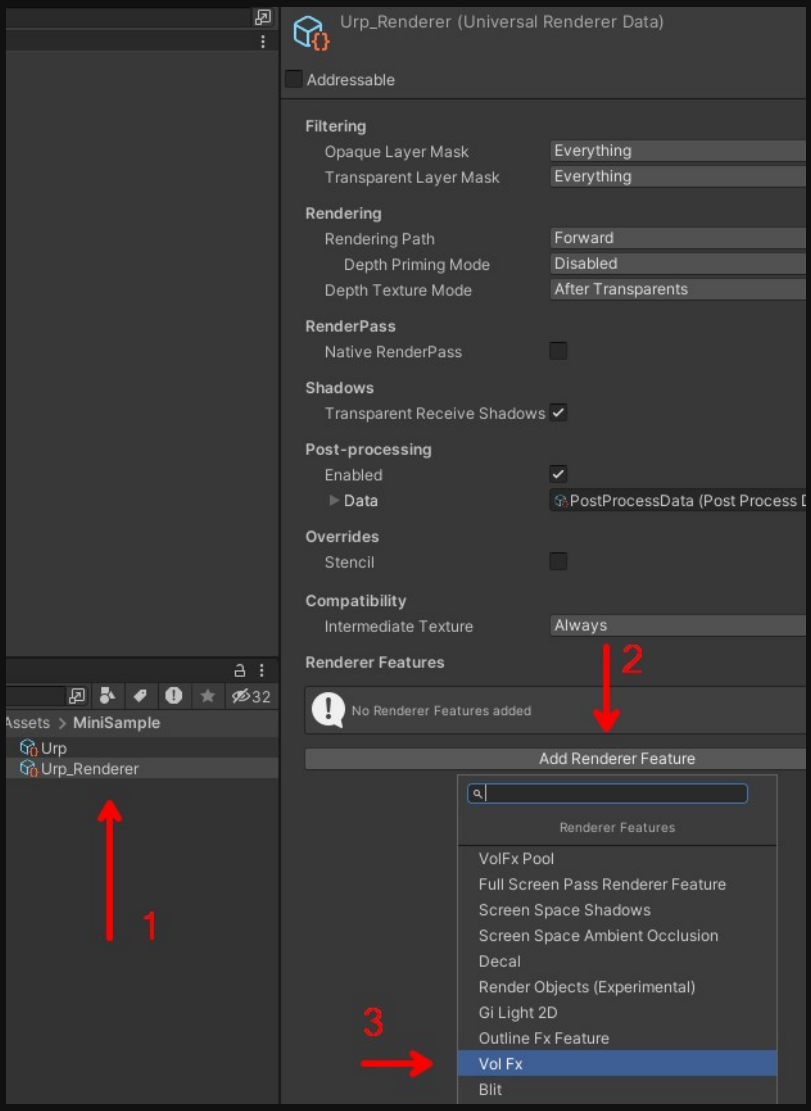
To create a processing module:

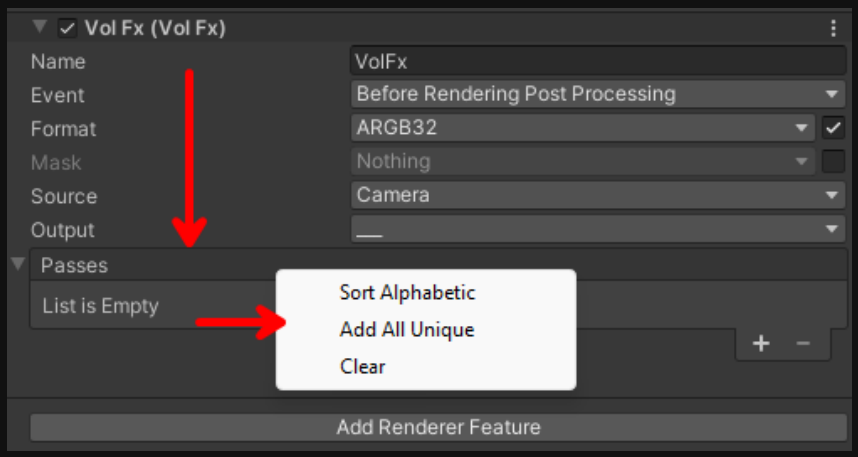
1. Add **VolFxRenderFeature** to the **URP Renderer**.
2. Configure the **pass sequence**.
3. Specify additional settings if needed.

Then:

4. Control them via **Volume on scene**.

1. To add a post-processing module, select the desired **URP Renderer** where you want the post-processing to run and click the **Add Render Feature** button.





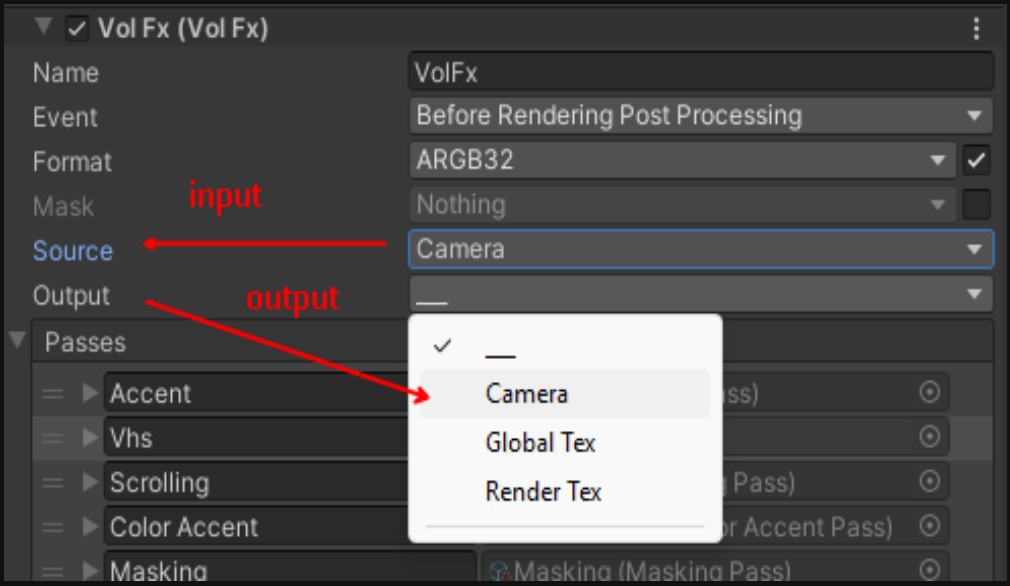
2. After that, you need to add Custom Passes. This can be done in two ways:

1. Click the **"+" button**.
2. Right-click on the **Passes** field and select **"Add All Unique."**

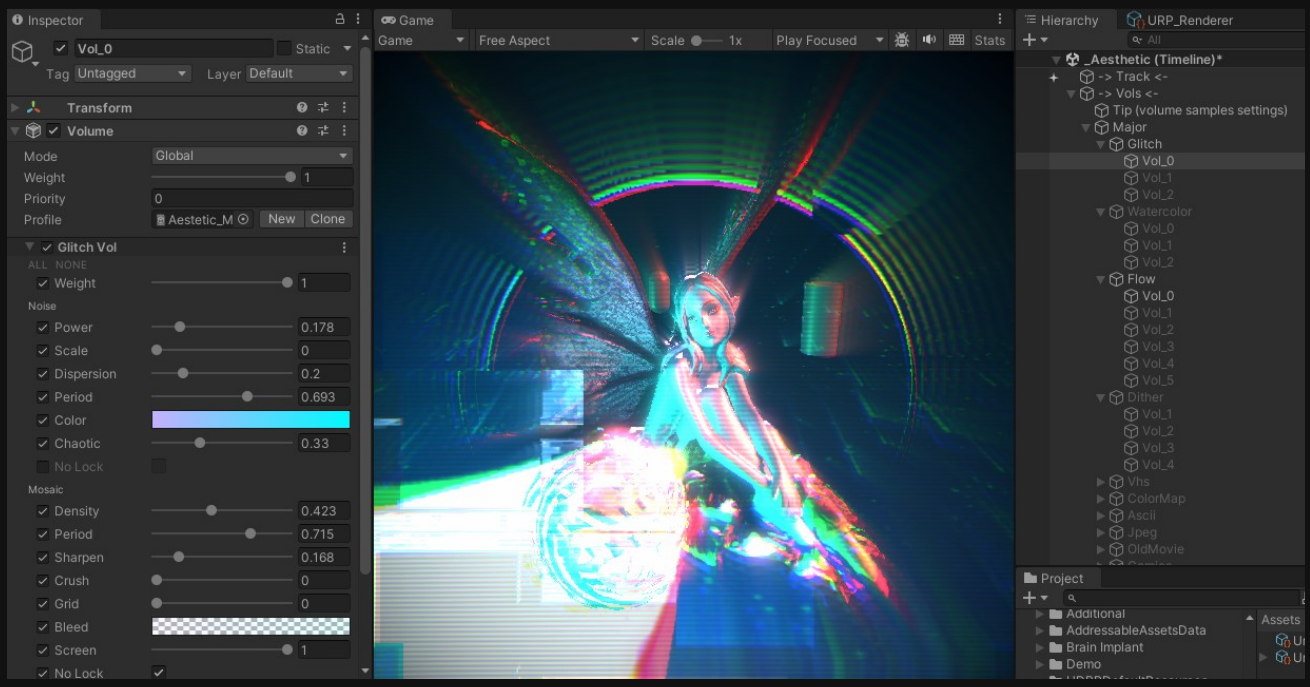
*** Important tip: Passes are executed in the same order as they appear in the list. To change the order of effects, you can rearrange them using Drag & Drop.**

3. Most module settings are simple and display a detailed description when you hover over them with the mouse.

* In simple terms, each module takes **input content (an incoming image)** and, after applying a series of active effects, outputs it to the selected target, such as the **current camera**, a **GlobalTexture**, or a **specified RenderTexture asset**.



After that, you can control **VolFx effects** the same way as regular post-processing by using **Volumes** in the selected scene.



Selective Application

To apply **VolFx** selectively to scene objects, you need to create and configure a separate module for this.

In the module settings:

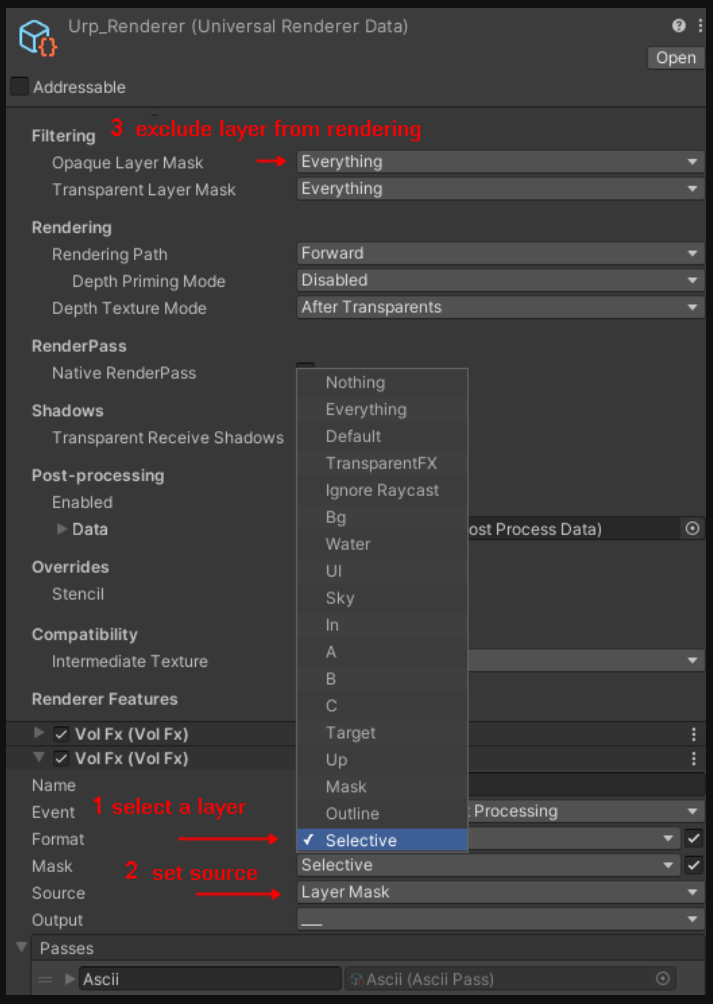
1. Specify the **Layer** you want the effects to be applied to.
2. Set the **source** to **LayerMask**.

Important:
Make sure to exclude those objects from the regular rendering pass using:

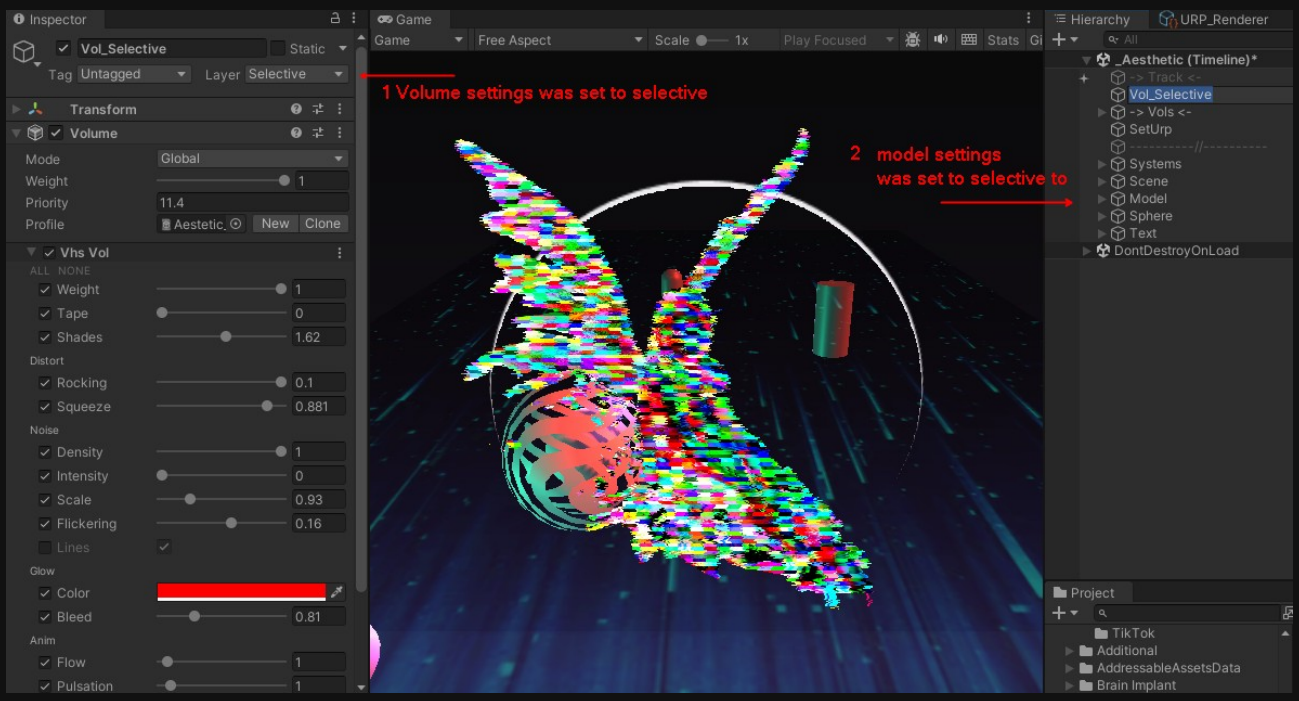
- **Filtering** in the **URP Asset**, or
- **Camera Culling** if you are using **URP 2D**

This ensures that the objects render correctly and are **not drawn twice**.

For more precise configuration, it is recommended to use the **Frame Debugger**.



> **After that**, you can use the **same Layer** in the scene for the **Volume** that controls the settings and for the **objects** the selective effect is applied to.



* The same approach can be applied to different input and output sources, such as global texture, render textures, VolFx pools, and of course camera content.

Creating a Custom Pass

To create a **CustomPass**, you need to inherit it from **VolFx.Pass** and override the required functions for **material setup** or **rendering control**. After that, it will appear in the list of available passes like a regular one that you can use in the post-processing chain.

```
[ShaderName("Hidden/VolFx/Grayscale")] // shader name for pass material
public class GrayscalePass : VolFx.Pass
{
    public override string ShaderName => string.Empty;

    // ========================================================================
    public override bool Validate(Material mat)
    {
        // use stack from feature settings, feature use custom VolumeStack with its own
LayerMask
        var settings = Stack.GetComponent<GrayscaleVol>();

        // return false if we don't want to execute pass, standart check
        if (settings.IsActive() == false)
            return false;

        // setup material before drawing
        mat.SetFloat("_Weight", settings.m_Weight.value);
        return true;
    }
}
```

By default, the material is created automatically using the **shader path and shader name**, taken from the **shader itself** (for example, from the shader declared in the shader file like *Shader "Hidden/VolFx/Grayscale"*). The material is then updated every time before processing is executed.

```
// called to perform rendering
public virtual void Invoke(CommandBuffer cmd, RTHandle source, RTHandle dest,
                           ScriptableRenderContext context,
                            ref RenderingData renderingData)
{
    Utils.Blit(cmd, source, dest, _material, 0, Invert);
}
```

In this way you can expand the engine and create dynamic effects controlled via VolumeProfile and scenes that have their own processing pipelines.